

课程笔记

课程笔记：Lecture 6B 强化学习 (Reinforcement Learning)

Codex

2026 年 4 月 16 日

Robots that Learn with **Reinforcement Learning**

CS 294-277 Guest Lecture

Haozhi Qi

1

视频作者/频道：Robots That Learn / UC Berkeley CS 294-277

发布日期：课程讲次日期：2024-10-14

视频时长：39:30

视频链接：<https://www.youtube.com/watch?v=pvTYmRXLixo>

目录

1 课程定位与本讲目标	2
1.1 本讲在课程中的位置	2
1.2 本章小结	2
2 Simulation Environments: 哪些模拟器在用, 为什么	2
2.1 MuJoCo、Raisim、Brax 与 Isaac Gym	2
2.2 速度不等于最好	3
2.3 本章小结	3
3 Sim2Real 的两大工具: Parallel Simulation 与 Domain Randomization	3
3.1 Parallel Simulation	3
3.2 Domain Randomization	3
3.3 本章小结	4
4 案例一: OpenAI dexterous in-hand manipulation	4
4.1 为什么这个案例重要	4
4.2 这个案例里怎么做 domain randomization	4
4.3 本章小结	4
5 案例二: High-speed quadrotor flight in the wild	5
5.1 为什么这个案例也能成立	5
5.2 四个设计选择	5
5.3 为什么讲者说 RL 不是唯一选项	5
5.4 本章小结	5
6 总结与延伸	5
6.1 值得记住的三点	5
6.2 拓展阅读	6
6.3 结束语	6

Robots that Learn with Reinforcement Learning

CS 294-277 Guest Lecture

Haozhi Qi

1

图 1: 课程官方材料中的代表性页面, 用于帮助读者在进入本讲正文前建立整体上下文。

1 课程定位与本讲目标

这一讲延续上一讲的 RL 主线, 但讨论重心明显从“算法是什么”移动到“怎样把算法真正用到机器人里”。讲者想回答的核心问题是: 如果我们已经知道 RL 可以学策略, 为什么机器人领域仍然如此依赖模拟器、并行仿真和 domain randomization (域随机化)?

1.1 本讲在课程中的位置

上一讲讲的是 RL 的基本定义、policy gradient 族和 simulator 的物理骨架; 这一讲则进一步进入实践层: 有哪些 simulators 被广泛使用, 它们各自适合什么任务, sim-to-real gap 到底如何缩小, 以及现实世界里有哪些代表性成功案例。

本讲的关键判断

在机器人 RL 里, “训练快”不是唯一标准。精度、可控性、可并行性、接口成熟度和部署便利性, 通常比单纯的仿真速度更重要。

1.2 本章小结

如果把上一讲看成“RL 的算法底座”, 那么这一讲就是“RL 的工程底座”。它关心的不只是模型能否学会, 而是模型能否跨过仿真与现实之间的鸿沟。

2 Simulation Environments: 哪些模拟器在用, 为什么

2.1 MuJoCo、Raisim、Brax 与 Isaac Gym

课程列举了几种学术界和工业界常见的 simulator:

- MuJoCo (Multi-Joint dynamics with Contact): 广泛使用的物理引擎, 擅长多刚体与接触建模, 背后以 C++ 实现, 近年来也有面向 GPU/TPU 并行的 MuJoCo XLA (MJX) 版本;

- Raisim: 偏 CPU 的刚体物理引擎, 适合机器人与 AI 研究;
- Brax: 用 JAX 写成、面向加速硬件的可微物理引擎, 适合单设备和多设备并行;
- Isaac Gym: NVIDIA 推出的 GPU-based simulator, 强调大规模并行仿真。

这些选择背后的共同点是: 它们都试图让机器人训练更快、更标准化、更可重复, 但它们侧重点不同。MuJoCo 偏通用且成熟, Brax 偏研究与可微分, Isaac Gym 偏并行和 GPU 吞吐, 而 Raisim 则常见于 CPU 侧的高精度刚体场景。

2.2 速度不等于最好

讲者特别提醒, simulator 不是越快越好。某些任务需要更高的数值精度, 或者对接触建模更敏感, 此时 CPU 方案反而可能比 GPU 更合适。换句话说, simulator 的选择应该由任务决定, 而不是由“谁跑得最快”单独决定。

不要把 simulator 只当成加速器

很多失败的机器人实验不是因为算法错了, 而是因为模拟器与任务不匹配: 接触模型、数值稳定性、精度和接口方式都会影响最终的策略质量。

2.3 本章小结

模拟器的价值不只在吞吐量, 更在于它是否能提供足够可信、足够稳定、足够可控的训练环境。对机器人 RL 来说, 训练平台本身就是方法的一部分。

3 Sim2Real 的两大工具: Parallel Simulation 与 Domain Randomization

3.1 Parallel Simulation

Parallel simulation 的作用是一次生成大量交互数据。课程强调, 这一点对 RL 尤其关键, 因为 RL 的学习通常依赖大量探索, 而单线程、单环境采样会让训练速度非常慢。并行仿真可以把多个环境同时跑起来, 从而显著提高样本生成速度。

讲者给出的直觉是: 如果一个环境只能顺序地产生经验, 那么策略改进会被物理世界的时间约束卡住; 如果能把训练搬到大量并行环境里, 就可以把“等机器人动起来”的时间转化成“算力在同一时刻处理更多轨迹”。

3.2 Domain Randomization

Domain randomization 的核心是: 在训练时故意打乱模拟环境的若干方面, 让策略见过更广的变化, 从而在真实世界里更容易泛化。课程把它分成两类:

- dynamics randomization: 随机化动力学参数, 如质量、摩擦、外力扰动;
- visual randomization: 随机化视觉因素, 如纹理、光照、相机位置、噪声和模糊。

这一思想本质上是在做“把现实世界的不确定性预先摊开”。如果真实机器人面对的参数会漂移，那么训练时就不要假设参数固定不变。

域随机化的直觉

不是让仿真更像某一个现实，而是让策略对一大类现实都“见怪不怪”。只要真实系统落在训练分布附近，策略就更可能稳定工作。

3.3 本章小结

Parallel simulation 解决“数据生成太慢”的问题，domain randomization 解决“模拟过于理想化”的问题。它们通常一起出现，因为一个负责规模，一个负责鲁棒性。

4 案例一：OpenAI dexterous in-hand manipulation

4.1 为什么这个案例重要

课程把 OpenAI 的灵巧手操作案例当成 sim-to-real 的代表性成功之一。原因很简单：dexterous in-hand manipulation（灵巧手内操作）是一个接触复杂、动力学高度耦合、现实误差很难忽略的任务。如果连这种任务都能从仿真迁移到现实，那么 sim-to-real 的方法论就有了很强说服力。

4.2 这个案例里怎么做 domain randomization

讲者列出的随机化内容非常具体：

- observation noise：给观测加入高斯噪声，同时加入相关噪声和非相关噪声；
- physics randomizations：对质量、摩擦、关节阻尼等参数进行采样；
- timing noise：通过改变每一步对应的物理仿真步数，模拟执行节奏变化；
- unmodeled effects：把仿真中没有显式建模的效果补进去，例如 tendon actuation 带来的 backlash（回程间隙）；
- visual appearance randomizations：改变灯光、相机、纹理和材料外观。

这里最值得注意的是，很多随机化并不是“加点噪声”这么简单，而是针对真实系统中最容易失配的环节逐项补偿。比如 backlash 不是视觉问题，而是机械传动的物理问题。

真实部署时最难的地方

灵巧操作的难点不在“会不会做动作”，而在接触、误差、时间尺度和传感器噪声同时出现时，策略还能不能保持稳定。

4.3 本章小结

OpenAI 灵巧手案例说明，sim-to-real 并不是把一个仿真策略直接扔到现实里，而是要通过系统性的随机化，把仿真中的假设尽量推进现实中的不确定性分布。

5 案例二：High-speed quadrotor flight in the wild

5.1 为什么这个案例也能成立

第二个案例是高速无人机在真实环境中的飞行。这个问题的特点是：决策必须快、视觉输入可能复杂、环境变化大，而且对在线推理延迟极其敏感。课程用它说明，sim-to-real 的关键不只在“把策略训出来”，还在“让感知与控制的接口足够轻量”。

5.2 四个设计选择

课程把这个案例的成功归纳为四点：

1. 使用 depth image（深度图）而不是 RGB 图像作为输入，降低视觉外观差异；
2. 采用轻量级神经网络，减少机载推理延迟；
3. 让网络输出多模态轨迹和碰撞代价，而不是单一动作；
4. 训练完成后 zero-shot 地直接部署到真实环境。

其中 depth representation 很关键，因为它对纹理、颜色和光照变化不那么敏感，更贴近“几何结构”而不是“外观”本身。多模态输出也很重要，因为高速飞行中的可行路径往往不止一条，强行平均反而会得到危险的中间解。

5.3 为什么讲者说 RL 不是唯一选项

这部分课件和讲解的一个微妙但重要的信息是：能成功并不意味着必须靠纯 RL。对于某些高速导航任务，感知设计、轨迹预测和损失函数设计同样关键。讲者强调这一点，是为了避免把 RL 神化成万能方案。

5.4 本章小结

无人机案例说明，sim-to-real 的成功常常来自一组组合拳：合适的输入表示、轻量模型、多模态输出、合理训练和足够鲁棒的部署策略，而不是单一技巧。

6 总结与延伸

这一讲可以总结为一句非常工程化的话：机器人 RL 想要真正落地，必须同时处理“算得快”和“迁得过去”这两件事。前者依赖并行仿真和合适的 simulator，后者依赖 domain randomization、输入表示设计和任务级的鲁棒性策略。

6.1 值得记住的三点

- 模拟器的选择取决于任务，而不是单纯取决于速度；
- parallel simulation 解决吞吐，domain randomization 解决失配；
- 高质量 sim-to-real 往往是系统设计，而不只是算法技巧。

6.2 拓展阅读

- *Learning Dexterous In-Hand Manipulation*
- *Learning agile and dynamic motor skills for legged robots*
- *Learning High-Speed Flight in the Wild*

6.3 结束语

如果把上一讲理解为“RL 作为算法”，这一讲就更像“RL 作为系统”。真正强的机器人系统通常不是只靠一个模型，而是靠 simulator、randomization、表示学习和部署约束共同组成的整体。